

Device Code Phishing: When MFA Is Not Enough

Exploiting OAuth 2.0 RFC 8628 Device Authorization Grant

Presenter

Seyed Mehdi Sahafi Najafi Nejad

Platform Identifiers

edX: seyyedmahdi-j2d0a3e | **GitHub:**
seyyedmahdinajafi

Timeline

Incident: March 2026 | **Recorded:** August
2026

Understanding RFC 8628

The OAuth 2.0 Device Authorization Grant specification designed for browserless and input-constrained environments.

Originally built to enable Smart TVs, IoT hardware, and CLI tools (e.g., Azure CLI) to obtain authorization using a secondary device with full browser capabilities.

Standard Flow vs Weaponized Threat

✔ Legitimate RFC 8628 Flow

User-Initiated: The user requests authentication from a Smart TV or terminal tool (e.g., az login).

Secondary Auth: User receives a 9-character code and completes login on their smartphone or PC browser.

Expected Access: Tokens are returned directly to the initiating legitimate device for session setup.

⚠ Weaponized Device Code Phishing

Attacker-Initiated: Threat actor executes background POST request to generate a device code pair.

Social Engineering: Victim is tricked into visiting official login.microsoft.com/devicelogin and entering code.

Token Interception: Upon victim completion, tokens drop straight into the attacker's polling terminal.

Anatomy of a Device Code Attack

Code Generation

Attacker sends POST to IdP endpoint using public client IDs (Office/Azure CLI) to request user_code & device_code.

Lure Delivery

Phishing message (email/Teams) tricks victim into approving access with the provided code on real IdP URL.

Victim Auth + MFA

Victim authenticates on legitimate Microsoft domain and completes standard MFA prompt. Zero fake domains involved.

Token Exfiltration

Attacker's daemon receives Access & Refresh tokens, hijacking session without ever capturing a password.

Why Traditional MFA Fails



100% Genuine Domain

Victims type their credentials directly into **login.microsoft.com**. Secure Web Gateways and domain reputation filters see clean, trusted traffic.



MFA Completion

The victim completes legitimate MFA for the attacker-initiated authentication request.



No Passwords Stolen

Skip credential harvesting entirely. The threat actor obtains OAuth Access & Refresh tokens directly via standard token endpoint polling.

Incident Analysis: March 2026 Breach



EvilTokens Automation: Surge of automated PhaaS toolkits leveraging RFC 8628 to bypass FIDO2 MFA en masse.



First-Party Client Abuse: Exploited immutable public Microsoft client IDs (Azure CLI, PowerShell) impossible to revoke globally.



Persistent Access: Requested `offline_access` scope, yielding refresh tokens that survived password resets and basic session revocations.



Limited Traditional Detection:

Because authentication occurs through legitimate identity infrastructure, traditional malware and domain-based detection may have limited visibility.



Identity Attack Vector Trends



Data indicates a 310% increase in OAuth Device Code Phishing across enterprise tenants between 2024 and 2026, driven by strict MFA enforcement neutralizing legacy phishing kits.

Attack Vector Comparison Matrix

Vector Attribute	Traditional Phishing	AiTM Proxy (Evilginx)	Device Code Phishing
Domain Hosted On	Spoofed / Typosquatted URL	Reverse Proxy Domain	Official login.microsoft.com
Captures Password?	Yes	Yes	No (Token Theft Only)
Bypasses MFA?	No	Yes	Yes (Pass-through)
Detection Level	Low (Easy to block)	Medium (Domain flags)	Extremely Difficult

Critical Severity Driver

0%

Malicious Infrastructure

Zero Malicious Artifacts to Scan

Because the attacker relies entirely on Microsoft's native authorization endpoints, traditional email gateways, DNS inspection, and endpoint detection agents detect **zero malicious links, binaries, or reverse proxy servers.**

"The ultimate stealth attack: forcing the victim to act as their own proxy directly on official identity infrastructure."

Defense & Mitigation Strategy



Conditional Access Policies (CAP): Explicitly block Device Code Flow in Entra ID / Okta for all non-administrative users or unmanaged devices.



Restrict OAuth App Consent: Disable user consent for apps requiring `offline_access` or high-privilege directory permissions.



Anomalous Token Monitoring: Query sign-in logs for non-interactive token refreshes originating from unexpected ASNs or impossible travel IPs.



User Security Training: Educate staff that device code prompts should *never* be entered unless actively initiating a login on a secondary device.

Questions?

Thank you for reviewing this final presentation.

CS50's Introduction to Cybersecurity

Final Project Submission



GitHub: seyyedmahdinajafi



edX: seyyedmahdi-j2d0a3e

Image Sources



<https://www.simplydata.com.my/wp-content/uploads/2026/05/ai-threat-hunting-dashboard-2026-05-23.png>

Source: www.simplydata.com.my